

GETTING STARTED WITH ESP8266 PROGRAMMING: Basic Arduino Programming

TS. DR. RAZIF BIN RAZALI

Arduino IDE Programming Language

The Arduino IDE primarily uses a programming language based on **C++**, with some elements of **C**.

It's often referred to as the "Arduino programming language" or "Wiring language," which essentially **simplifies C/C++** by providing a **set of easy-to-use functions and libraries** specifically designed for microcontroller interactions (like `digitalWrite()`, `analogRead()`, etc.).

So, while you are writing code that looks very much like C++, the Arduino IDE handles some pre-processing and includes libraries that make it more accessible for beginners and for quick prototyping.

Comments in Arduino IDE

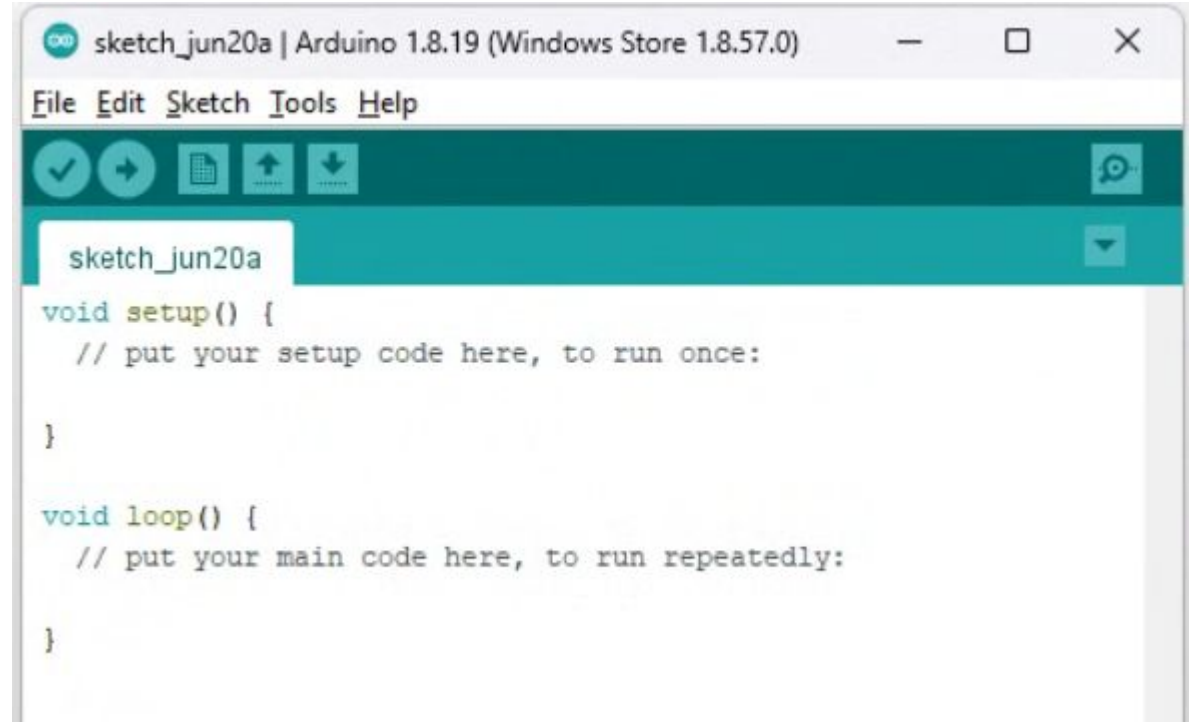
```
void setup() {  
  Serial.begin(9600); // Initialize serial communication at 9600 baud  
  pinMode(LED_BUILTIN, OUTPUT); // Set the built-in LED pin as an output  
}  
  
void loop() {  
  digitalWrite(LED_BUILTIN, HIGH); // Turn the LED on (HIGH is the voltage level)  
  delay(1000); // Wait for a second (1000 milliseconds)  
  digitalWrite(LED_BUILTIN, LOW); // Turn the LED off by setting the voltage level to LOW  
  delay(1000); // Wait for another second  
}
```

```
/*  
  Sketch Name: Blinky Example  
  Description: This sketch blinks the built-in LED on an Arduino board.  
               It's a classic "Hello World" for microcontrollers.  
  Author: Your Name  
  Date: June 21, 2025  
*/  
  
void setup() {  
  Serial.begin(9600);  
  pinMode(LED_BUILTIN, OUTPUT);  
}  
  
void loop() {  
  digitalWrite(LED_BUILTIN, HIGH);  
  delay(1000);  
}
```

Structure of Arduino Sketch

The **setup()** function is called when a sketch starts. Use it to initialize the variables, pin modes, start using libraries, etc. The setup function will only run once, after each power up or reset of the Arduino board.

the **loop()** function does precisely what its name suggests, and loops consecutively, allowing your program to change and respond. Use it to actively control the Arduino board.



Data types

void	Boolean	char	Unsigned char
long	Unsigned long	short	float
byte	int	Unsigned int	word
double	array	String-char array	String-object

Control Structures

control structures are fundamental for determining the flow of your program. They allow your program to make decisions, repeat actions, and execute specific **blocks of code** based on certain conditions.

```
if (someCondition) {  
    // do stuff if the condition is true  
}
```

```
if (someCondition) {  
    // do stuff if the condition is true  
} else {  
    // do stuff if the condition is false  
}
```

```
if (condition1) {  
    // do stuff if condition1 is true  
} else if (condition2) {  
    // do stuff if condition1 is false AND condition2 is true  
} else {  
    // do stuff if both condition1 and condition2 are false  
}
```

```
for (initialization; condition; increment/decrement) {  
    // code to be executed repeatedly  
}
```

```
while (someCondition) {  
    // code to be executed repeatedly as long as someCondition is true  
}
```

Digital I/O

The ESP8266 provides multiple General Purpose Input/Output (GPIO) pins that can be configured as either digital inputs or digital outputs.

Key Characteristics:

- **Logic Levels:** Operates on 3.3V logic. A HIGH signal is typically around 3.3V, and a LOW signal is 0V.
- **Input Modes:**
 - **INPUT:** Reads the state of an external component (e.g., button, sensor).
 - **INPUT_PULLUP:** Internally pulls the input pin HIGH, useful for active-LOW buttons or switches, preventing floating states.
- **Output Mode:**
 - **OUTPUT:** Drives a pin HIGH (3.3V) or LOW (0V) to control external components (e.g., LEDs, relays).
- **Important Considerations:**
 - Not all physical pins are available for general use; some are reserved for internal functions (e.g., boot mode, flash).
 - Avoid using GPIO 6-11 as they are connected to the integrated SPI flash.
 - GPIO 0, 2, and 15 have specific roles during boot-up; incorrect states can prevent the ESP8266 from booting.

GPIO Pin	D-Pin	Notes
GPIO1	TX	Default UART TX, can be used as GPIO
GPIO3	RX	Default UART RX, can be used as GPIO
GPIO0	D3	Boot-mode selection (LOW to enter flash mode)
GPIO2	D4	Boot-mode selection (HIGH for normal boot)
GPIO4	D2	General Purpose
GPIO5	D1	General Purpose
GPIO12	D6	General Purpose
GPIO13	D7	General Purpose
GPIO14	D5	General Purpose
GPIO15	D8	Boot-mode selection (LOW for normal boot)
GPIO16	D0	Deep Sleep Wake-up Pin, can be used as digital I/O

Analog I/O

The ESP8266 has a single Analog-to-Digital Converter (ADC) pin, typically labeled **A0** on development boards like the NodeMCU. This pin allows the microcontroller to measure analog voltage levels and convert them into a digital value.

Key Characteristics:

- **ADC Resolution:** The ESP8266's ADC has a 10-bit resolution, meaning it can convert analog voltages into digital values ranging from 0 to 1023 ($2^{10}-1$).
- **Input Voltage Range:** The default input voltage range for the ADC is 0 to 1.0V.
 - **CRITICAL:** Directly connecting voltages higher than 1.0V (e.g., 3.3V from a sensor) to the **A0** pin will damage the ESP8266.
 - To read higher voltages (e.g., 0-3.3V), an external **voltage divider** circuit is required to scale the input voltage down to the safe 0-1.0V range.
- **Usage:** Commonly used to interface with analog sensors that output a continuous range of voltage, such as:
 - Temperature sensors (e.g., LM35, thermistors)
 - Light sensors (e.g., photoresistors)
 - Potentiometers
 - Analog joysticks

Example (with Voltage Divider):

To read a 0-3.3V sensor output, you might use a voltage divider with resistors R_1 and R_2 such that:

$$V_{out} = V_{in} \times \frac{R_2}{R_1 + R_2}$$

For example, using $R_1 = 220k\Omega$ and $R_2 = 100k\Omega$:

$$V_{out} = V_{in} \times \frac{100k\Omega}{220k\Omega + 100k\Omega} = V_{in} \times \frac{100}{320} \approx V_{in} \times 0.3125$$

This scales a 3.3V input to approximately 1.03V, which is close to the 1.0V limit. Adjust resistor values for precise scaling.

GETTING STARTED WITH ESP8266 PROGRAMMING:

1st ESP8266 Program - Blinking an LED

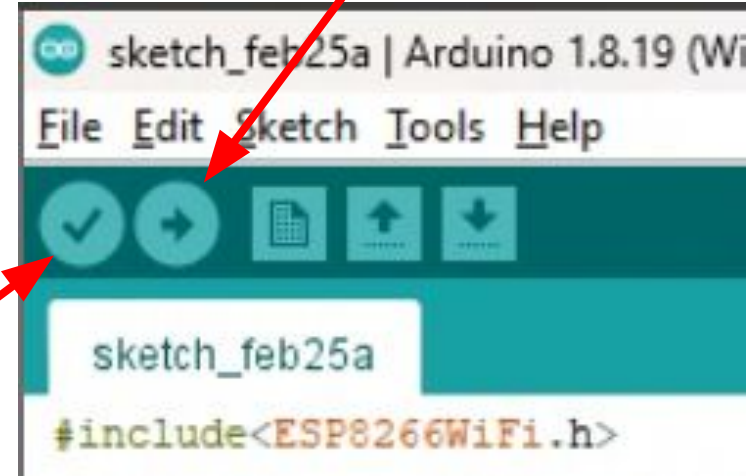
TS. DR. RAZIF BIN RAZALI

Code to blink the onboard LED

- copy code from [github](#), edit the code, debug and test,
- Uploading the sketch to the ESP8266.
- Troubleshooting common issues (port not found, upload errors).

upload the code to
microcontroller

to verify code



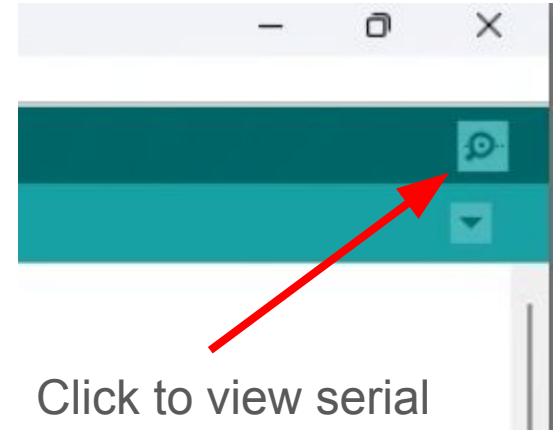
GETTING STARTED WITH ESP8266 PROGRAMMING: Connecting to WiFi

TS. DR. RAZIF BIN RAZALI

Connecting to WiFi

- Copy code from [github](#),
- Edit code to connect to WiFi,
- Make sure your laptop/pc and ESP8266 connect to the same WiFi network.
- Test connection, should Displaying connection status and IP address on the Serial Monitor.
- Turn ON and OFF onboard LED using URL based.(use any web browser)
- debug and test.

*If your friends connect with same network, try controlling their onboard LED from your laptop/pc.



Click to view serial monitor and select correct baud rate.